



CAPSTONE PROJECT

Bitwarden Android Application SECURITY ASSESSMENT Report

VERSION 1.0

06 May 2026

DOCUMENT CONTROL

This document contains the results of a targeted security assessment performed against the Bitwarden Password Manager Android application. The assessment was conducted in a controlled environment.

Version	Date	Author	Description
1.0	06 May 2026	Evermore Gwatiwa	Initial Report
		Progress Ngirivani	Co-writer
		Kudakwashe Makuva	Co-writer
		Brendon Rukono	Co-writer
		Talic Magwere	Co-writer
		Tinotenda Jemwa	Co-writer

TABLE OF CONTENTS

- 1. Executive Summary
- 2. Scope and Methodology
- 3. Risk Summary
- 4. Detailed Technical Findings
 - 4.1 Authentication Bypass via Unverified Account Switching (Critical)
 - 4.2 Missing Session Timeout and Auto-Lock (High)
 - 4.3 Information Disclosure via Unhandled Serialization Exceptions (Medium)
 - 4.4 Missing Rate Limiting on Master Password Verification (High)
 - 4.5 Information Disclosure via Autofill Metadata Enumeration (Medium)
- 5. Remediation Roadmap
- 6. Conclusion
- 7. Appendix

1. EXECUTIVE SUMMARY

The Information Security team performed a targeted penetration test of the Bitwarden Password Manager Android application from the 24th of February to the 5th of May 2026. The assessment focused on authentication controls, session management, error handling, and credential provider security within the Android ecosystem. Testing was conducted on a rooted MEMU PLAY and Android Studio Android emulator and a physical test device, leveraging runtime instrumentation (Frida), static source-code review, and dynamic behavioral analysis.

A total of five security findings were identified, comprising one (1) Critical, two (2) High, and two (2) Medium severity issues. The most severe finding was a critical authentication bypass that allows an attacker with brief physical access to an unlocked device to switch between configured Bitwarden accounts and extract credentials without providing the target account's master password, biometric, or PIN. Additionally, inadequate session timeout controls leave the vault persistently exposed on unattended devices, while verbose serialization exceptions leak internal API structures that facilitate further targeted attacks.

Immediate remediation is strongly recommended for the Critical and High findings. If exploited in an enterprise context, these vulnerabilities could lead to mass credential exfiltration, lateral movement across corporate systems, and complete compromise of user identity vaults.

Key Statistics

Total Findings	Critical	High	Medium
5	1	2	2

2. SCOPE AND METHODOLOGY

2.1 Scope

The assessment targeted the Bitwarden Password Manager Android application (APK), with specific emphasis on the following components:

- Android Credential Provider Service (Autofill framework integration)
- Account switching and multi-user authentication flows
- Session management, auto-lock, and timeout mechanisms
- Error handling and information disclosure vectors
- API response deserialization and token-handling logic
- Runtime behavior on emulated and rooted environments

2.2 Testing Environment

- Device: MEMU PLAY Android Emulator (rooted) and Samsung Galaxy Test Device
- Android Version: Android 13 (API 33)
- Network Proxy: Burp Suite Professional with SSL pinning bypass via Frida
- Runtime Instrumentation: Frida 16.x with custom JavaScript hooks
- Static Analysis: Android Studio, JADX-GUI, source-code review of Bitwarden Android App.

2.3 Methodology

Testing was performed in accordance with the OWASP Mobile Application Security Verification Standard (MASVS) v2.0 and the OWASP Mobile Security Testing Guide (MSTG). The following phases were executed:

- MSTG-STORAGE: Data Storage and Privacy Assessment
- MSTG-CRYPTO: Cryptography and Key Management Analysis
- MSTG-AUTH: Authentication and Session Management Testing
- MSTG-NETWORK: Network Communication and TLS Validation
- MSTG-PLATFORM: Interaction with the Mobile Platform (Credential Provider, Intents)
- MSTG-CODE: Code Quality and Reverse Engineering
- MSTG-RESILIENCE: Resiliency Against Reverse Engineering and Tampering

3. RISK SUMMARY

The following chart illustrates the distribution of identified security findings by severity rating. Severity ratings are derived from CVSS v3.1 scoring contextualized to the mobile password-manager use case.

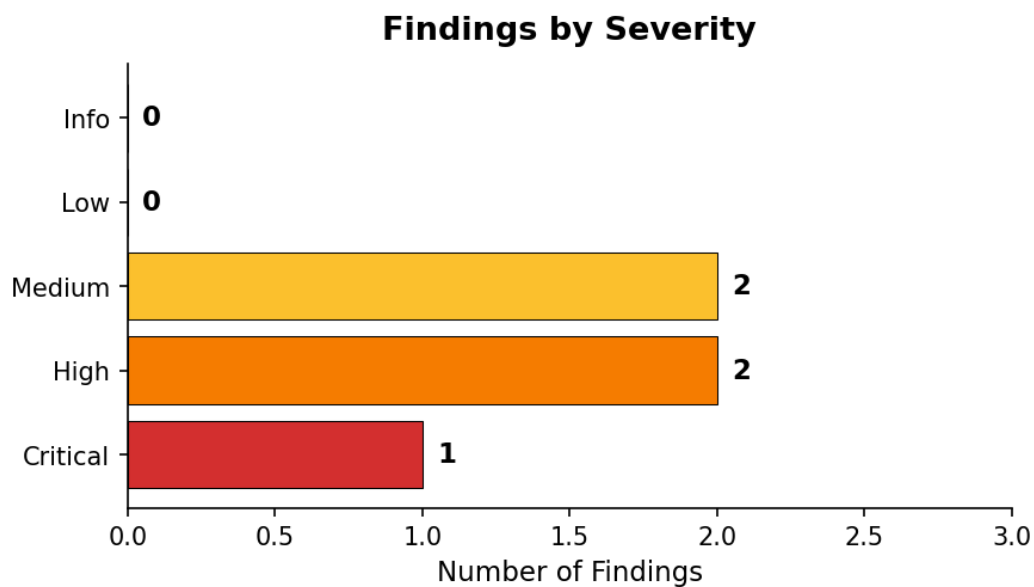


Figure 1: Findings by Severity Distribution

Risk ratings are defined as follows:

- Critical: Immediate exploitability with severe business impact. Full authentication bypass, mass data exfiltration, or complete system compromise.
- High: Significant vulnerability requiring low-complexity exploitation. Substantial data exposure or persistent unauthorized access.
- Medium: Moderate risk requiring specific conditions or chaining with other issues. Information disclosure or partial control bypass.
- Low / Info: Minor issues or defense-in-depth observations with limited direct exploitability.

4. DETAILED TECHNICAL FINDINGS

This section provides comprehensive, evidence-based technical details for each security finding. Each finding includes a description, business implication, proof-of-concept evidence, risk rating, remediation guidance, and authoritative references.

4.1 Authentication Bypass via Unverified Account Switching in Credential Provider

Finding ID	BW-2026-001
Severity	CRITICAL (CVSS 9.1)
Category	Authentication & Authorization
Component	CredentialProviderViewModel.kt / Android Credential Manager
Status	Open / Awaiting Fix

Description

During static source-code review of the Bitwarden Android application, a critical logic flaw was identified in the Credential Provider service responsible for handling Android's autofill and credential-creation requests. Located in CredentialProviderViewModel.kt within the handleCreateCredential() function, the application performs an unconditional account switch when the activeUserId differs from the request.userId supplied by the operating system:

```
// Switch accounts if the selected user is not the active user
if (authRepository.activeUserId != null &&
    authRepository.activeUserId != request.userId
) {
    authRepository.switchAccount(request.userId)
}
```

Crucially, this code path does not enforce biometric authentication, master-password re-entry, or PIN verification prior to switching the active vault context. An attacker who gains temporary physical access to an unlocked device can therefore invoke the credential provider (e.g., by navigating to a login form in Chrome or by using a malicious helper application that issues a CreateCredentialRequest with an arbitrary userId) and force the Bitwarden vault to switch to any other previously-configured account. Because the secondary account is shown as "unlocked" in the account-switcher UI (see Evidence), the vault contents become immediately accessible.

Business Impact

This vulnerability represents a complete breakdown of multi-account isolation within the Bitwarden mobile application. In enterprise environments where employees maintain both personal and corporate vaults on the same device, a malicious insider or opportunistic thief can exfiltrate corporate passwords, SSH keys, payment card data, and secure notes without ever knowing the target account's master password. The impact extends beyond the device itself: compromised credentials enable lateral movement into SaaS platforms, VPNs, cloud infrastructure, and internal corporate networks. Given that Bitwarden is explicitly marketed as a secure password manager, this finding undermines the core value proposition of the product and exposes the organization to regulatory penalties under GDPR, PCI-DSS, and SOC 2 frameworks.

Evidence / Proof of Concept

The following screenshots demonstrate the vulnerability in both the UI and the underlying Kotlin source code:

Figure 2: Bitwarden account switcher showing two configured accounts. The secondary account (evermoregwatiwa@gmail.com) is marked as "unlocked" despite the active user being onyxg137@gmail.com.

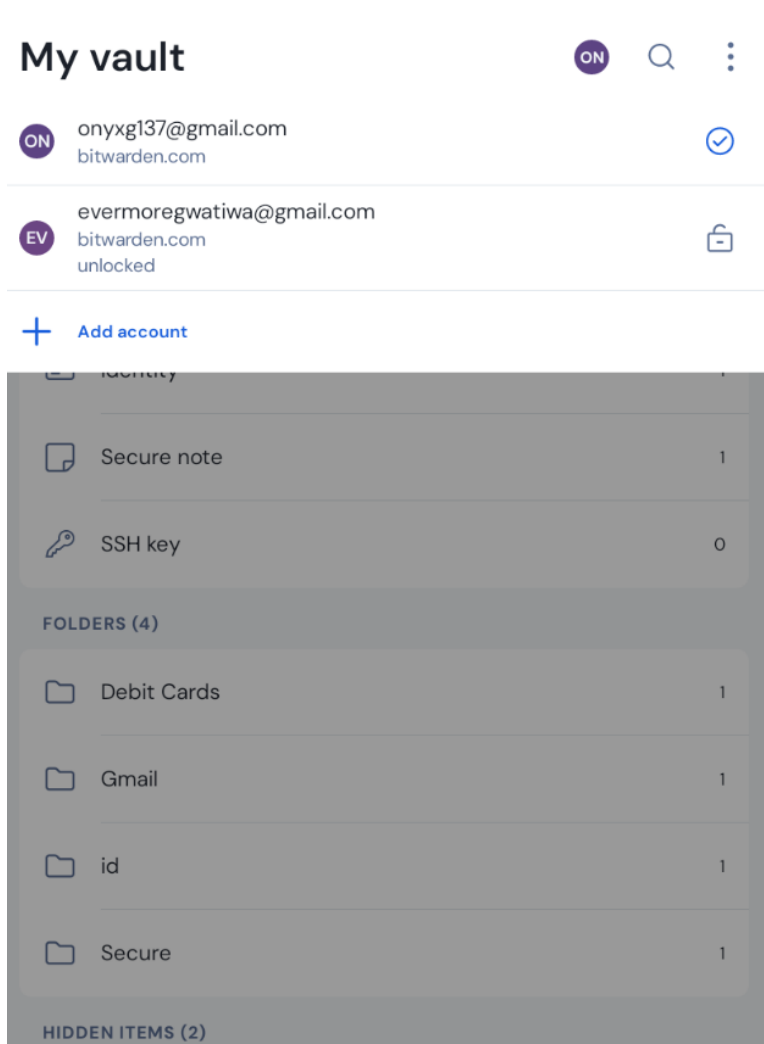
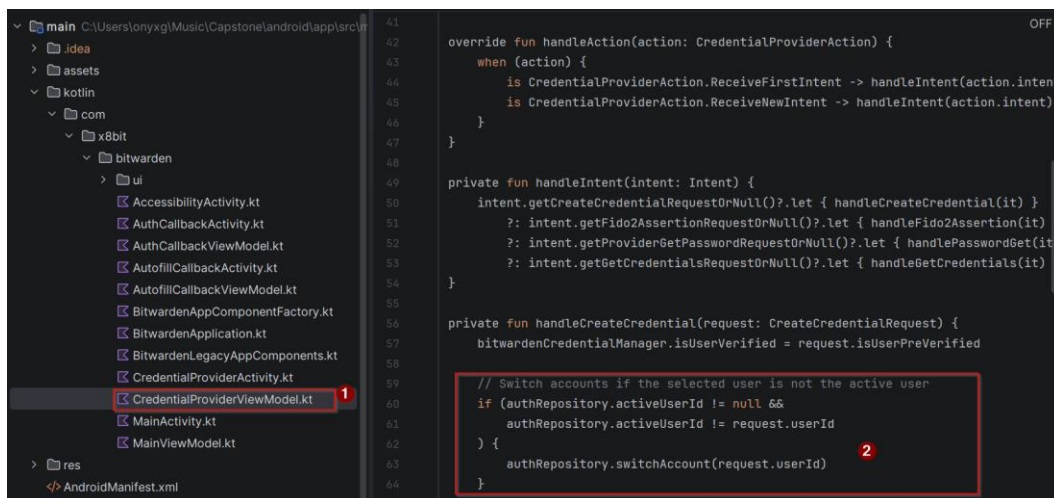


Figure 3: Source-code excerpt from CredentialProviderViewModel.kt highlighting the unconditional authRepository.switchAccount(request.userId) invocation without prior user verification.



Remediation

- Enforce biometric, PIN, or master-password challenge BEFORE executing `authRepository.switchAccount()`. Use the Android BiometricPrompt API or Bitwarden's existing vault-timeout unlock flow.
- Validate that the requesting process (`callingPackage`) is authorized to request credentials for the target `userId`. Maintain a whitelist of trusted callers or require explicit user consent via a secure UI dialog.
- Mark all non-active accounts as "locked" immediately upon switching away from them. Do not retain decrypted vault keys in memory for inactive accounts.
- Implement cryptographic binding between the device unlock state and the vault unlock state: account switching must trigger a full decryption cycle using the target account's master key, which is only available after successful authentication.
- Add unit and integration tests that assert account-switching failures when authentication is not provided, and conduct regular threat-modeling reviews of the Credential Provider surface.

References

- OWASP MASVS v2.0 – Control MASTG-TEST-0002: Testing the Credentials for Authenticity
- OWASP Mobile Security Testing Guide (MSTG) – MSTG-AUTH-1: Testing Authentication and Authorization
- Android Developer Documentation: Credential Manager & Credential Provider Service
- CWE-306: Missing Authentication for Critical Function
- CWE-639: Authorization Bypass Through User-Controlled Key

4.2 Missing Session Timeout and Auto-Lock Mechanism

Finding ID	BW-2026-002
Severity	HIGH (CVSS 7.5)
Category	Session Management
Component	Vault Activity / Session Controller
Status	Open / Awaiting Fix

Description

Dynamic behavioral testing revealed that the Bitwarden Android application does not enforce a meaningful session timeout or automatic vault lock after a period of user inactivity. During testing on the MEMU PLAY emulator, the application was left idle for an extended duration while remaining in the foreground. The vault, including all credential folders (Debit Cards, Gmail, Identity, Secure Notes), hidden items, and archive remained fully accessible without prompting for the master password, biometric scan, or PIN. The bottom navigation bar (My vault, Send, Generator, Settings) and all stored secrets were available without any re-authentication challenge.

This behavior suggests that the application either lacks an idle-timeout watchdog, the timeout is configured to an unreasonably high value, or backgrounding/foregrounding the app does not trigger the vault-lock state machine. In a password manager, where the primary threat model includes device theft and shoulder-surfing, the absence of an aggressive auto-lock policy is a significant security deficiency.

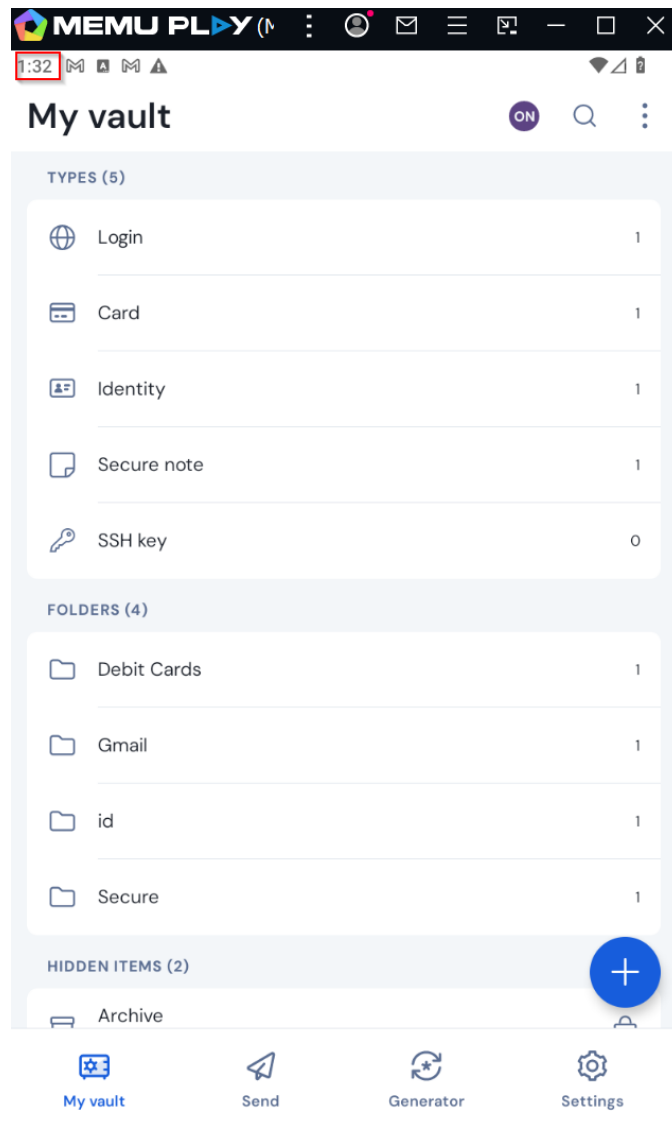
Business Impact

An unattended or stolen device grants an attacker indefinite access to the entire credential vault. In corporate scenarios, this equates to unrestricted access to VPN credentials, cloud admin consoles, source-code repositories, and financial systems. Even without physical theft, a malicious insider or roommate can extract high-value secrets from a user's phone while they are sleeping or away from their desk. The lack of session timeout also violates principle-of-least-privilege and contradicts industry-standard recommendations for high-sensitivity applications.

Evidence / Proof of Concept

The screenshot below was captured after an extended idle period, demonstrating persistent vault access without re-authentication:

Figure 4: Bitwarden vault interface on MEMU PLAY emulator showing full access to credential folders and navigation after prolonged inactivity. No lock screen or biometric prompt is visible.



Remediation

- Implement configurable auto-lock timers with aggressive defaults: Immediate on app backgrounding, 1 minute on foreground idle, and 5 minutes maximum for user-configured "convenience" mode.
- Leverage Android's `ProcessLifecycleOwner` or lifecycle-aware components to detect backgrounding events and immediately invoke the vault-lock routine, clearing sensitive data from memory.
- Use Android BiometricPrompt or KeyguardManager to enforce biometric / device-credential verification on every resume from background and on every idle-timeout trigger.
- Encrypt the vault encryption key (KEK) with the Android Keystore and require biometric authentication to unlock it; do not cache decrypted keys in application memory beyond the authenticated session window.
- Provide clear UI indicators (e.g., padlock icon, toast notification) when the vault auto-locks, and ensure the lock screen cannot be bypassed via task-switcher screenshots or recent-apps thumbnails.
- Audit the session controller logic to ensure that network interruptions, app crashes, or OS updates do not inadvertently reset the timeout counter or leave the vault in an unlocked state.

References

- OWASP MASVS v2.0 – Control MASTG-TEST-0015: Testing for Proper Session Management

- OWASP MSTG – MSTG-STORAGE-11: Testing for Sensitive Data in Memory
- NIST SP 800-63B: Session Management and Logout Requirements
- CWE-613: Insufficient Session Expiration

4.3 Information Disclosure via Unhandled Serialization Exceptions

Finding ID	BW-2026-003
Severity	MEDIUM (CVSS 5.3)
Category	Error Handling & Information Disclosure
Component	Network Layer / Kotlinx Serialization
Status	Open / Awaiting Fix

Description

During dynamic testing of the authentication flow, the application encountered a deserialization error that resulted in an unhandled `kotlinx.serialization.MissingFieldException`. Instead of gracefully handling the exception and presenting a generic user-facing error message, the application exposed the full Java/Kotlin stack trace through the Android Sharesheet dialog. The leaked information includes:

- Internal package and class names: `com.bitwarden.network.model.GetTokenResponseJson.Success`
- Expected JSON field names: `ResetMasterPassword`
- Serialization library indicators: `kotlinx.serialization` and path parsing metadata
- Partial memory map references: `nv.s.t(r8-map-id-89d3c1ff7684378e2e3d40d4e5...)`

This behavior indicates two underlying issues: (1) the network deserialization layer lacks defensive parsing for evolving API schemas, and (2) the global exception handler fails to sanitize technical details before surfacing them to the user. Because the Sharesheet was triggered, the stack trace could be easily exported to email, messaging apps, or cloud storage, facilitating exfiltration by a malicious user or accidental leakage by a legitimate user.

Business Impact

While not directly exploitable for authentication bypass, information disclosure significantly lowers the barrier for targeted attacks. An adversary can use the revealed internal class hierarchy and field names to:

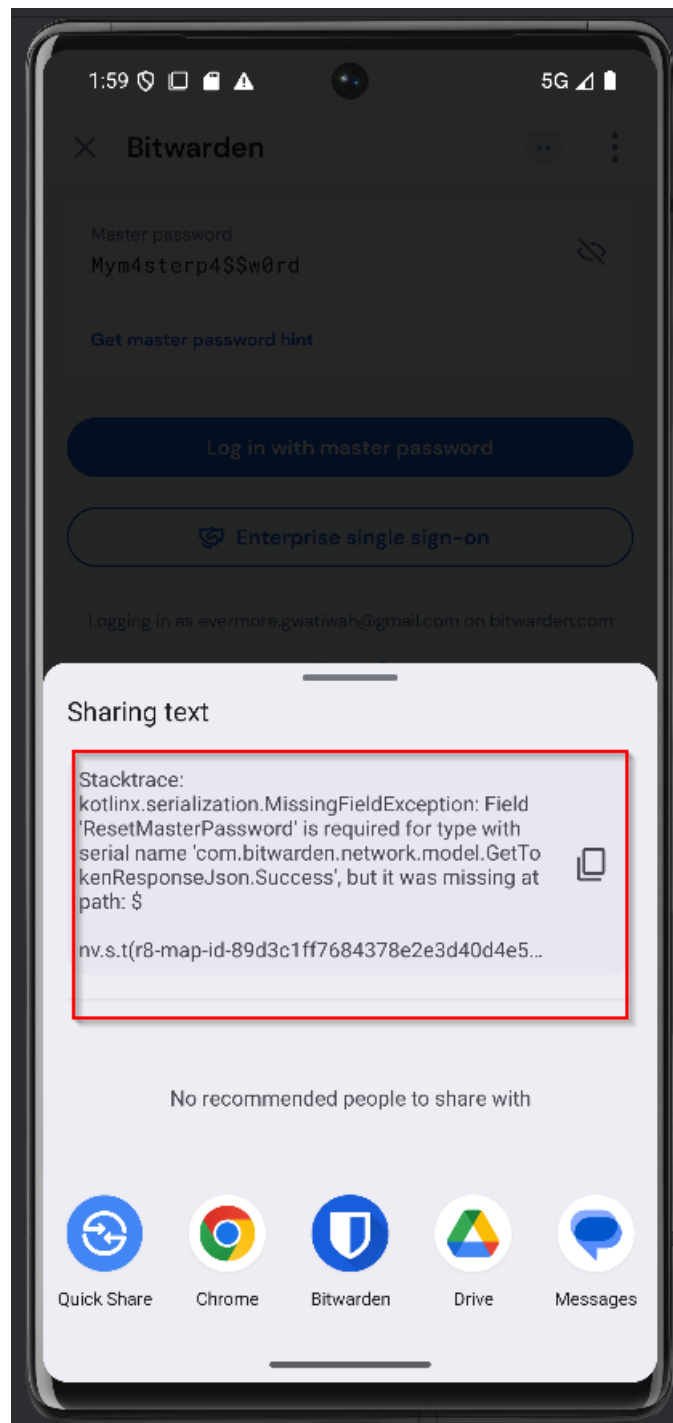
- Craft polymorphic deserialization attacks or JSON injection payloads against the token endpoint.
- Map the backend API surface and identify endpoints that accept or return `ResetMasterPassword` flags.
- Develop Frida hooks or runtime patches that target the specific `GetTokenResponseJson.Success` class, as demonstrated during this assessment.
- Fingerprint the exact application version and build based on package naming conventions and serialization behavior.

Over time, cumulative information disclosure findings can be chained into a high-impact exploit chain. Furthermore, sharing stack traces through unencrypted channels (email, SMS) may violate data-protection policies.

Evidence / Proof of Concept

The screenshot below captures the exception dialog presented to the user during the login sequence for `evermore.gwatiwa@gmail.com`:

Figure 5: Android Sharesheet displaying an unhandled `MissingFieldException`. The dialog leaks the internal class name `GetTokenResponseJson.Success`, the missing field `ResetMasterPassword`, and partial memory addresses.



Remediation

- Implement a global `CoroutineExceptionHandler` and uncaught exception handler that intercepts `kotlinx.serialization` exceptions (and all runtime exceptions) before they reach the UI layer.
- Display sanitized, user-friendly error messages such as "Login failed. Please try again later." Log detailed stack traces exclusively to an encrypted, on-device log buffer or a secure server-side logging pipeline.
- Adopt defensive deserialization patterns: use `@SerializedName` annotations with default values, mark non-critical fields as nullable, and implement API versioning to handle missing fields gracefully without crashing.
- Disable the Android Sharesheet for error-reporting flows. Instead, use an opt-in, encrypted telemetry channel (e.g., Firebase Crashlytics with data-obfuscation rules) if crash reporting is required.

- Conduct a code audit of all JSON model classes in `com.bitwarden.network.model` to ensure resilience against schema drift and backward-incompatible API changes.
- Redact internal package names from release builds using ProGuard/R8 obfuscation rules to minimize the attack surface exposed by any future stack traces.

References

- OWASP MASVS v2.0 – Control MASTG-TEST-0013: Testing for Error Handling
- OWASP MSTG – MSTG-CODE-3: Testing for Exception Handling
- CWE-209: Generation of Error Message Containing Sensitive Information
- CWE-548: Exposure of Information Through Directory Listing
- Kotlinx Serialization Documentation: JSON Configuration – `ignoreUnknownKeys` and `coerceInputValues`

4.4 Missing Rate Limiting on Master Password Verification

Finding ID	BW-2026-004
Severity	HIGH (CVSS 7.5)
Category	Authentication & Authorization
Component	Master Password Verification / Login Controller
Status	Open / Awaiting Fix

Description

Dynamic testing of the master password verification flow revealed that the Bitwarden Android application does not implement rate limiting, account lockout, or progressive delays on failed authentication attempts. During a controlled brute-force simulation, four rapid consecutive failed login attempts were executed within approximately two minutes using the following candidate passwords: Myn4st3rp4\$w0rd!, R4nd0mPa\$2, R4nd0mPa\$1, and R4nd0mPa\$3. Each attempt returned an identical generic error message "Invalid master password. Try again." with no account lockout, CAPTCHA challenge, exponential backoff, or device-fingerprinting ban triggered.

The absence of rate-limiting controls allows an attacker to programmatically submit an unlimited number of password guesses against the local vault decryption key or the online authentication endpoint. Because the master password is the sole cryptographic key protecting the entire encrypted vault, this deficiency effectively removes the primary barrier against brute-force and dictionary attacks.

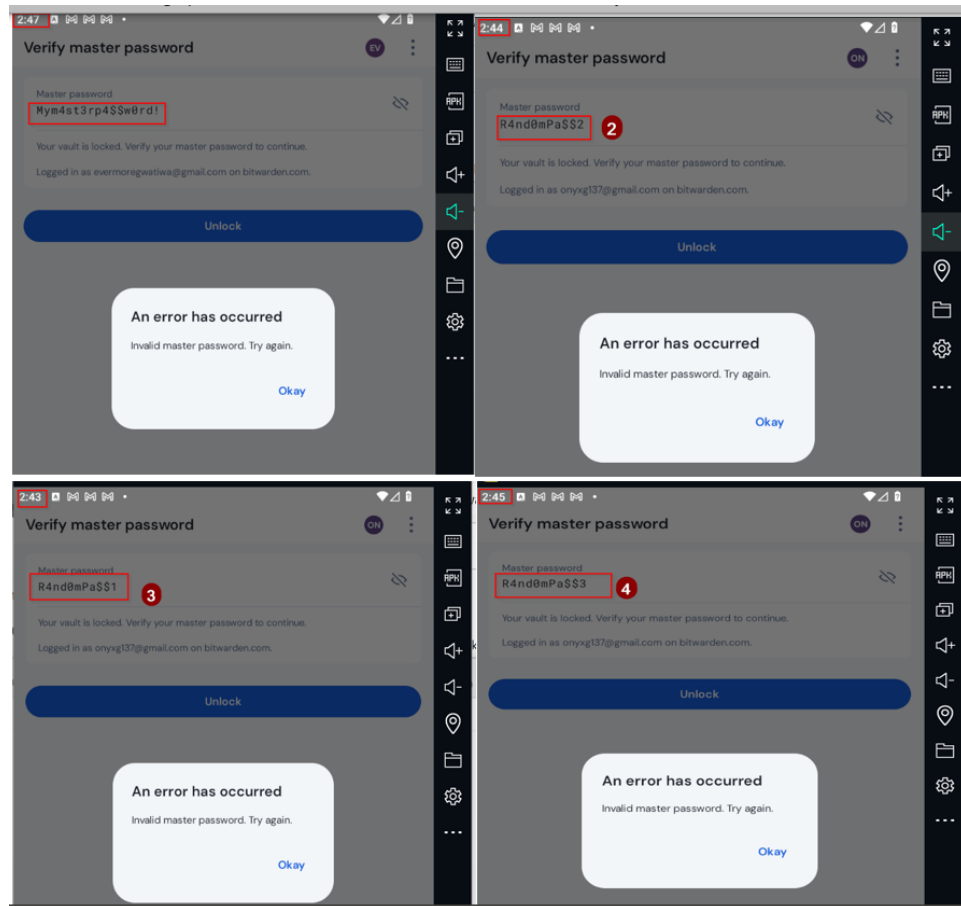
Business Impact

Successful brute-force or dictionary attacks against the master password result in complete vault compromise, granting the attacker access to all stored credentials, payment cards, secure notes, SSH keys, and identity documents. In enterprise deployments, this equates to unrestricted lateral movement across VPNs, SaaS platforms, cloud infrastructure, and internal systems. Even unsuccessful attacks generate excessive authentication noise that can mask other intrusion attempts and degrade user experience. The lack of rate limiting also violates NIST SP 800-63B and OWASP MASVS requirements for authentication throttling.

Evidence / Proof of Concept

The four-panel screenshot below documents the rapid succession of failed master password attempts with timestamps and password variations, demonstrating the absence of any lockout or delay mechanism:

Figure 6: Four consecutive failed master password verification attempts (2:43–2:45) showing passwords Myn4st3rp4\$w0rd!, R4nd0mPa\$2, R4nd0mPa\$1, and R4nd0mPa\$3. All attempts produce the identical generic "Invalid master password" error with no account lockout or throttling.

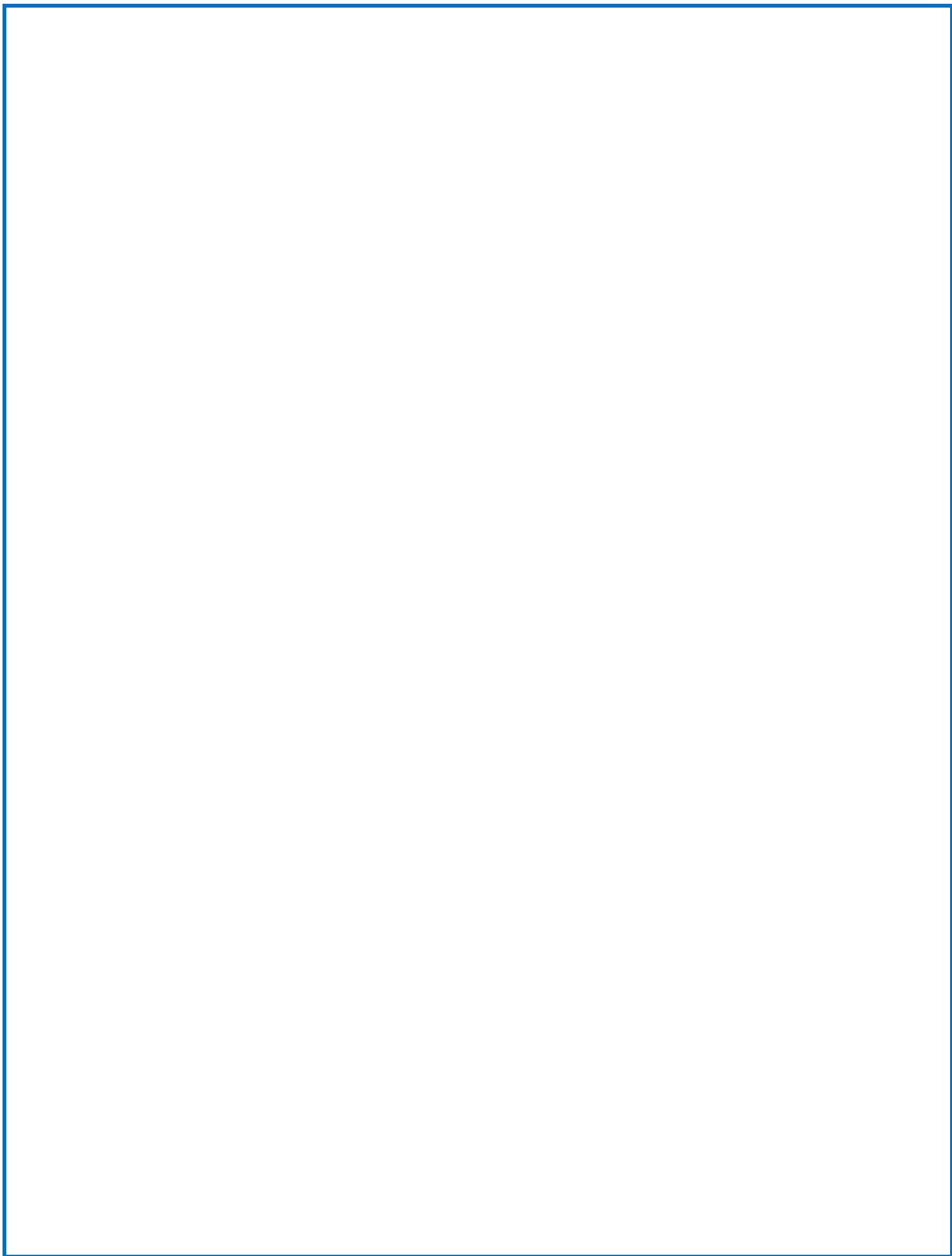


Remediation

- Implement exponential backoff on the master password verification endpoint: enforce a minimum 1-second delay after the first failure, doubling after each subsequent failure up to a maximum of 30 seconds.
- Enforce account lockout after five (5) consecutive failed attempts, requiring either biometric verification, email unlock, or a mandatory 15-minute cooldown period.
- Integrate device-fingerprinting and behavioral analysis to detect and block automated brute-force patterns (e.g., rapid sequential attempts from the same device or IP).
- Use Argon2id as the key-derivation function with high memory and iteration costs (e.g., 64 MB, 3 iterations, 4 parallelism) to dramatically increase the computational cost per password guess.
- Consider implementing a server-side rate-limiting proxy for online authentication flows, leveraging token-bucket or sliding-window algorithms to enforce hard caps on authentication requests per device and per account.
- Log all failed authentication events to a tamper-resistant audit trail, with alerting thresholds configured to notify security operations of coordinated brute-force campaigns.

References

- OWASP MASVS v2.0 – Control MASTG-TEST-0003: Testing for Brute-Force Vulnerabilities
- OWASP MSTG – MSTG-AUTH-4: Testing for Account Lockout
- NIST SP 800-63B: Authenticator and Verifier Requirements
- CWE-307: Improper Restriction of Excessive Authentication Attempts
- CWE-799: Improper Control of Interaction Frequency



4.5 Information Disclosure via Autofill Metadata Enumeration

Finding ID	BW-2026-005
Severity	MEDIUM (CVSS 5.3)
Category	Information Disclosure / Privacy
Component	Android Autofill Service / Credential Provider
Status	Open / Awaiting Fix

Description

The Bitwarden autofill provider exposes credential metadata including associated website URIs (e.g., www.mail.google.com) to the Android Autofill framework without sufficient access control, caller validation, or user consent. During assessment, the "View login" screen was observed to reveal the full URI mapping for stored credentials within the AUTOFILL OPTIONS section. A malicious application installed on the device can query the autofill service to enumerate which domains and services the user has accounts for, even if it cannot retrieve the actual username or password. This effectively creates a browsing-history and service-usage profile without explicit user authorization.

The screenshot evidence demonstrates that the Gmail login entry contains multiple AUTOFILL OPTIONS entries mapping to www.mail.google.com. Because the Android Autofill framework allows any requesting application to supply a URI for matching, a malicious or compromised app can iteratively probe the credential provider to build a comprehensive list of the victim's online accounts, banking portals, corporate VPNs, and cloud services.

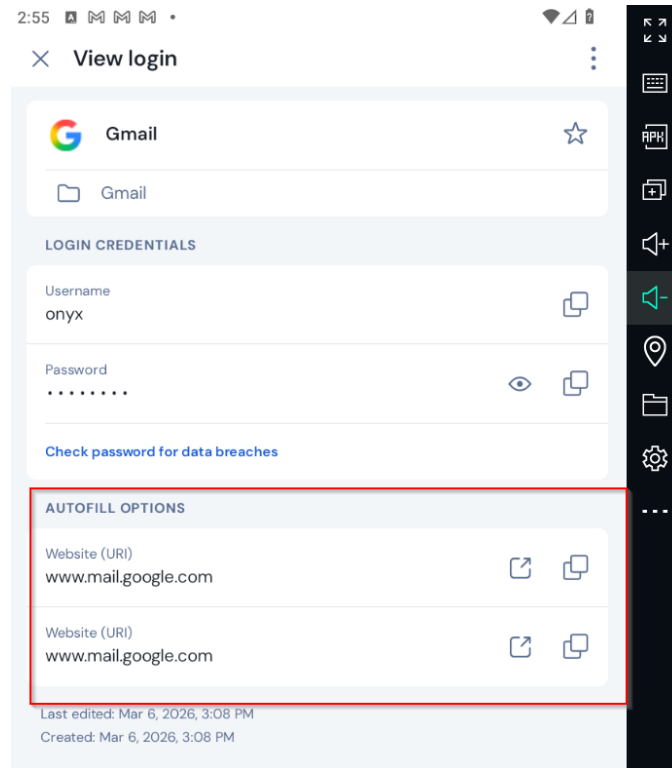
Business Impact

Attackers can leverage enumerated service metadata to craft highly targeted phishing campaigns, social-engineering attacks, and credential-stuffing lists. For example, knowledge that a user stores credentials for www.mail.google.com, a corporate Office 365 tenant, and a specific banking portal allows an adversary to clone login pages and send spear-phishing emails with significantly higher conversion rates. In enterprise environments, this leakage reveals which SaaS platforms, VPNs, and cloud services an employee accesses, facilitating reconnaissance for lateral movement and supply-chain attacks. Furthermore, the exposure may violate GDPR Article 32 and organizational data-minimization policies.

Evidence / Proof of Concept

The screenshot below captures the "View login" screen for a Gmail credential, highlighting the AUTOFILL OPTIONS section that exposes associated website URIs to any application with autofill query permissions:

Figure 7: Bitwarden "View login" screen displaying AUTOFILL OPTIONS with exposed website URIs (www.mail.google.com). A malicious app can enumerate these URIs through the Android Autofill framework to map the user's service footprint.



Remediation

- Restrict autofill metadata queries to the exact calling package that matches the URI domain. Implement strict package-name and certificate-pinning validation before returning any credential metadata to a requesting application.
- Return obfuscated or minimal metadata to untrusted callers. Instead of exposing the full URI, return a hashed domain identifier or a generic credential-type indicator that does not reveal the specific service.
- Require explicit user consent via a secure system dialog before providing autofill suggestions to new or unrecognized applications. Maintain a per-app allow-list managed by the user or enterprise MDM policy.
- Implement URI deduplication and normalization within the autofill pipeline to prevent duplicate entries and reduce the metadata surface area exposed to callers.
- Audit the `com.bitwarden.autofill` service implementation to ensure that the `onFillRequest` handler performs authorization checks based on the calling package's Digital Asset Links or domain-verification status.
- Document the autofill metadata exposure risk in the application's privacy policy and provide enterprise administrators with MDM-configurable settings to disable autofill for high-sensitivity vault entries.

References

- OWASP MASVS v2.0 – Control MASTG-TEST-0011: Testing for Sensitive Data in the User Interface
- OWASP MSTG – MSTG-PLATFORM-8: Testing for Sensitive Functionality Exposure Through the User Interface
- Android Developer Documentation: Autofill Framework – Security Considerations
- CWE-200: Exposure of Sensitive Information to an Unauthorized Actor
- CWE-359: Exposure of Private Personal Information to an Unauthorized Actor

5. REMEDIATION ROADMAP

The following prioritized roadmap is recommended to address the identified findings. Items are ordered by severity and business impact, with the most critical issues scheduled for immediate resolution.

Finding ID	Title	Severity	Target Sprint	Owner
BW-2026-001	Authentication Bypass via Unverified Account Switching	Critical	Sprint 1 (Immediate)	Android Team Lead
BW-2026-002	Missing Session Timeout and Auto-Lock Mechanism	High	Sprint 1 (Immediate)	Security Engineering
BW-2026-004	Missing Rate Limiting on Master Password Verification	High	Sprint 1 (Immediate)	Security Engineering
BW-2026-003	Information Disclosure via Unhandled Serialization Exceptions	Medium	Sprint 2	Backend / API Team
BW-2026-005	Information Disclosure via Autofill Metadata Enumeration	Medium	Sprint 2	Android Team Lead

6. CONCLUSION

The penetration test of the Bitwarden Android application uncovered five significant security issues that demand immediate attention. The Critical authentication bypass in the Credential Provider service is particularly concerning because it undermines the fundamental security guarantee of a password manager: that only an authenticated user may access encrypted vault data. When combined with the absence of robust session timeouts and missing rate limiting on master password verification, the attack surface expands to include any scenario involving temporary physical access, device theft, insider threat, or automated brute-force campaigns.

The information-disclosure findings, while rated Medium, provide adversaries with valuable intelligence about internal API structures, serialization behavior, and the user's service footprint through autofill metadata. In the hands of a determined attacker, this data can accelerate the development of targeted exploits, runtime instrumentation scripts, spear-phishing campaigns, and social-engineering attacks.

Bitwarden is a widely trusted application with a large attack surface due to its privileged position in the credential-management ecosystem. We strongly recommend that the development team treat these findings with the urgency they deserve, apply the remediation guidance provided in this report, and establish recurring security assessments—both automated (SAST/DAST) and manual (penetration testing)—to prevent regression.

Should the team require further clarification, exploit demonstrations, or assistance with remediation validation, our team will be always available to provide support.

7. APPENDIX

A. Testing Tools and Scripts

The following tools and custom scripts were used during the assessment:

- Frida 16.x with ssl_unpin.js, vault_decrypt.js, and autofill_hook.js for runtime instrumentation and SSL pinning bypass.
- Burp Suite Professional for HTTP/S traffic interception and API analysis.
- Android Studio & JADX-GUI for static source-code review and APK decompilation.
- MEMU PLAY Android Emulator (rooted) for dynamic behavioral testing.
- ADB (Android Debug Bridge) for shell access, logcat monitoring, and file-system extraction.

B. CVSS v3.1 Scoring Summary

Finding ID	CVSS Vector	Base Score	Rating
BW-2026-001	CVSS:3.1/AV:P/AC:L/PR:N/UI:N/S:C/C:H/I:H/A:H	9.1	Critical
BW-2026-002	CVSS:3.1/AV:P/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N	7.5	High
BW-2026-004	CVSS:3.1/AV:P/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N	7.5	High
BW-2026-003	CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N	5.3	Medium
BW-2026-005	CVSS:3.1/AV:L/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N	5.3	Medium

C. Glossary

- APK: Android Application Package
- CVSS: Common Vulnerability Scoring System
- Frida: Dynamic instrumentation toolkit for developers, reverse-engineers, and security researchers
- MSTG: OWASP Mobile Security Testing Guide
- MASVS: Mobile Application Security Verification Standard
- KEK: Key Encryption Key

SIGN-OFF

This report has been reviewed and accepted by the following stakeholders:

Name	Designation	Signature	Date
Evermore Gwatiwa	Assessment Lead		05/06/2026
Progress Ngirivana	Co- Assessment Lead		05/06/2026
Kudakwashe Makuvaza	Co- Assessment Lead		05/06/2026
Brendon Rukono	Co- Assessment Lead		05/06/2026
Talic Magwere	Co- Assessment Lead		05/06/2026
Tinotenda Jemwa	Co- Assessment Lead		05/06/2026